

Creating and Managing Unit Files in Systemd



Andrew Mallett

Author and Trainer

@theurbanpenguin | www.theurbanpenguin.com



Overview



Systemd Services

- Investigating startup speed
- Provisioning loop devices
- Creating service units to support loop devices
- Working with timer units
- Working with socket units
- Troubleshooting services





System Bootup Time



```
$ systemd-analyze  
$ systemd-analyze blame  
$ systemd-analyze dump
```

Systemd-Analyze

One of the really useful systemd command is `systemd-analyze`, allowing you to view the time taken in booting the system



Demo



Investigate System Boot
- systemd-analyze





Service Units



**Rather than scripting
startup of services, systemd
service unit files allow using
an INI formatted file**



```
$ sudo fallocate -l10G /root/disk1  
$ MYDISK=$(sudo losetup -f /root/disk1 --show)  
$ sudo parted $MYDISK mklabel msdos mkpart primary 0% 100% print  
$ lsblk
```

Loop Devices

Loop devices are commonly used in Ubuntu and link raw disk files to device files. However, on a reboot the devices will not show. To persist them, you can create a service unit.




```
$ sudo systemctl edit --full --force disk1.service
[Unit]
Description=Loop device
After=local-fs.target
[Service]
Type=oneshot
ExecStart=/sbin/losetup /dev/loop3 /root/disk1
ExecStart=/sbin/partprobe /dev/loop3
[Install]
WantedBy=multi-user.target
$ sudo systemctl enable disk1
```

Service Unit

Adding the two required commands to the service unit will ensure that the disk file is linked and the partitions are read. Make sure you implement the correct loop device in your file.



Demo



Create Disk Service Unit



Timer Units - The New Cron



Timer units run service units on a schedule allowing far greater reporting than traditional cron jobs



```
$ systemctl list-timers
```

List Timers

Straight away you can see the difference between timer units and traditional cron jobs. You can see the next scheduled execution as well as the last execution time. Additionally, no extra daemon is required. Systemd runs all the time.




```
$ sudo systemctl edit --full --force touchfile.timer
[Unit]
Description=Touch File Timer
[Timer]
OnBootSec=60s
OnUnitActiveSec=600s
RandomizedDelaySec=300s
[Install]
WantedBy=timers.target
```

Timer Unit

By default the timer unit will trigger a service unit with the same name. The touchfile.timer unit will trigger the touchfile.service. It is scheduled to happen 60 seconds after boot and then every 10 minutes. To add randomization to reduce services running concurrently, the schedule will adjust between 10 and 15 minutes adding a 5 minutes randomization.



Demo



Timer Units:

- Create timer unit
- Create scheduled service





Socket Units



Sockets allow services to be started when users connect to a port on the server. Systemd holds the port open and starts the service



```
$ systemctl list-sockets  
$ sudo apt install -y cockpit  
$ systemctl list-sockets  
$ sudo ss -ntlp 'sport = :9090'  
$ curl localhost:9090  
$ sudo ss -ntlp 'sport = :9090'
```

List and Activate Sockets

You can list sockets in much the same way as timers. The web management interface cockpit, works with sockets. Systemd holds the port open, and the web management interface is started when users connect to the socket.



```
$ sudo mkdir /web ; echo hello | sudo tee /web/index.html ; cd /web  
$ python3 -m http.server &> /dev/null &  
$ curl localhost:8000  
$ kill %1
```

Simple Python Web Server

Python has a simple web server module you can use. It defaults to listen on TCP Port 8000. The kill command can terminate the given job number of the backgrounded task.




```
$ sudo systemctl edit --full --force web.socket  
[Unit]  
Description=Python HTTP Server Socket  
[Socket]  
ListenStream=8080  
[Install]  
WantedBy=sockets.target
```

Socket Unit

The simple Python web server is not written to interact with systemd socket activation. The socket will listen on Port 8080 and will start the web server on Port 8000, a different port. This is used simply to demonstrate sockets. The beta Apache 2.5 web server supports socket units



Demo



Socket Units:

- Show Cockpit socket
- List sockets
- Create socket and activate service



```
$ sudo journalctl -u ssh  
$ sudo journalctl -u ssh -g vagrant  
$ sudo systemctl status ssh
```

Troubleshoot Services

Journalctl is excellent at searching the systemd journal log. The option `-u` can be used to search for a unit file, adding the option `-g` allows searching for a regular expression. The simple status of `systemctl` is useful to view how long the service has been running



Demo



Troubleshoot Units:

- Journalctl
- Systemctl



Summary



Working with Systemctl

- Systemd-Analyze
 - blame
 - dump
- Service Units
 - Disk files
- Timer Units
- Socket Units
- Troubleshooting

Up Next:

Working with OpenSSL

